

Reduce Log Ingestion Costs

with Pattern Detection in Splunk

Michele Caci

Senior Software Engineer @ Amadeus





Michele Caci

Senior Software Engineer @ Logging team in Amadeus

Focus

Logging - OpenTelemetry

Stack

Go · K8S - Splunk

Hobbies

Cooking, Languages, Board Games

We log too much

And if you're here, you probably log too much as well

~50 TB

of logs ingested per day at Amadeus

Billions

events generated daily

Hundreds of applications, pipelines, and services

These logs are expensive

```
2026-06-30T08:42:01.004Z DEBUG [db-pool] trx=8f3a2
Acquiring connection pool=main
2026-06-30T08:42:01.006Z DEBUG [db-pool] trx=8f3a2
Connection acquired latency=2ms
2026-06-30T08:42:01.008Z INFO [booking-svc] trx=8f3a2
Validating passenger data fields=12
2026-06-30T08:42:01.009Z DEBUG [validator] trx=8f3a2
Field check: name OK
2026-06-30T08:42:01.009Z DEBUG [validator] trx=8f3a2
Field check: dob OK
2026-06-30T08:42:01.010Z DEBUG [validator] trx=8f3a2
Field check: passport OK
2026-06-30T08:42:01.011Z DEBUG [cache] trx=8f3a2 Cache
miss key=passenger:8f3a2
2026-06-30T08:42:01.013Z DEBUG [db-pool] trx=8f3a2
Acquiring connection pool=main
2026-06-30T08:42:01.014Z DEBUG [db-pool] trx=8f3a2
Connection acquired latency=1ms
2026-06-30T08:42:01.021Z INFO [pricing-svc] trx=8f3a2
Fetching fare rules origin=NCE
2026-06-30T08:42:01.023Z DEBUG [http-client] trx=8f3a2
GET /fare-rules timeout=5000ms
2026-06-30T08:42:01.038Z DEBUG [http-client] trx=8f3a2
```

The Problem With Logging Everything



High Cost

Splunk licenses on **ingestion volume**.

Every DEBUG line, every heartbeat, every redundant stack trace...
You pay for all of it.



Low Signal

When everything is logged, **nothing stands out**.

Finding a real error means scanning through thousands of irrelevant lines.



Rarely Read

Logs are mostly read when something goes wrong, they **usually stay ignored** otherwise.

You're storing and indexing data that will rarely be read.

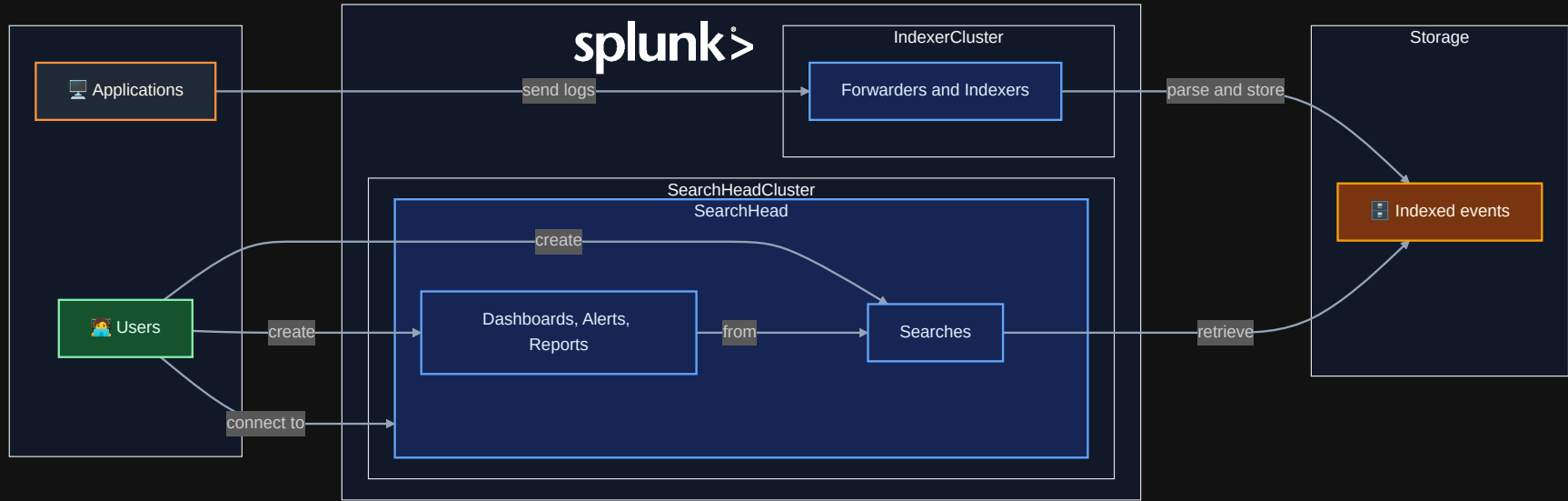
THE GOOD NEWS

You don't need to log less.
You need to log smarter.

And **pattern detection with Splunk** is only one of the available tools from the box

Overview of Splunk

- **Indexing flow** enables application events to become searchable insight
- **Search flow** enables fast searches across huge volumes of events



💡 Savings are made at indexing time. At search time you are left with cost analysis.

How to make savings at
indexing

Migrate from unstructured to structured logging

Change type: Application code change

Unstructured Logs

- Event is a free text to be parsed
- Event field extraction is **delegated** to Splunk
- **High CPU usage** on Splunk indexing flow

Structured Logs

- **Minimize** the event **payload**, **maximize** event **field** usage
- Event field extraction is easier on Splunk
- **Save CPU usage** on Splunk indexing flow
- Indexing is agnostic to event structure

```
2026/07/02 23:00:14 measurements ok remote address is 192.168.1.99:3041 temperature is 29.4C humidity is 70% RH
```

```
time=2026-07-02T23:00:14.960+02:00 level=INFO msg="measurements ok" remote=192.168.1.99:3041 temperature=29.4 temperature_unit=C humidity=70% humidity_unit=RH
```

Implement your own filtering and extraction logic

Change type: Splunk configuration change

props.conf

- Create event and field manipulation **rules**
- Create routing **rules**

transforms.conf

- Apply props.conf manipulation **rules**
- **Route** events according to props.conf rules

```
[sensor_transactions]
# Timestamp parsing - matches "2026/07/02 23:00:14"
TIME_FORMAT      = %Y/%m/%d %H:%M:%S
TIME_PREFIX      = ^\s*
# Field extractions (search-time, via transforms.conf)
TRANSFORMS-sensor_fields = extract_sensor_fields
# Indexing routing (evaluated in order, first match wins)
# Both transforms are defined in transforms.conf
TRANSFORMS-routing = route_ok_to_sensor_ok,\
route_errors_to_sensor_errors
# Auto-assign sourcetype by file path
[source::.../sensor_*.log]
sourcetype = sensor_transactions
```

```
[extract_sensor_fields]
REGEX = ^\d{4}/\d{2}/\d{2}\s+\d{2}:\d{2}:\d{2}\s+(?P<transaction_id>\S+)
WRITE_META = true # field extraction at index time

[route_ok_to_sensor_ok]
REGEX = (?i)\b\w+\s+ok\b
DEST_KEY = _MetaData:Index
FORMAT = sensor_ok

[route_errors_to_sensor_errors]
REGEX = (?i)(?:error|fail|timeout|refused|denied|critical|alert|warn)
DEST_KEY = _MetaData:Index
FORMAT = sensor_errors
```

What to do after indexing

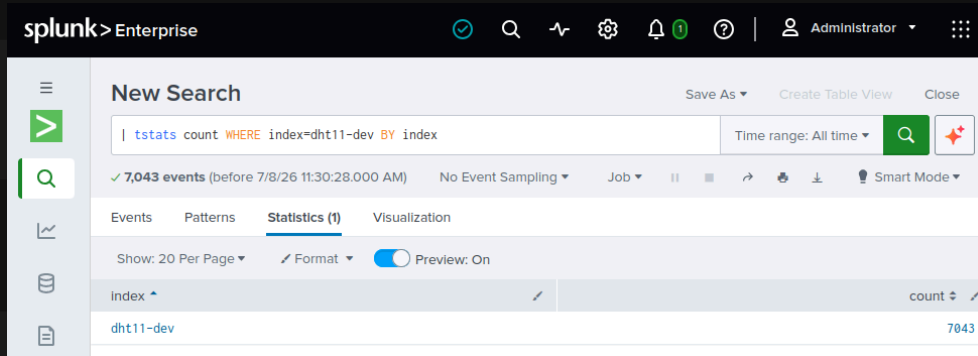
Query #1: General view of how your application logs

- Get the event count

```
| tstats count  
  WHERE index=your_index_1 OR index=your_index_2  
  BY index
```

- More fields for a more granular view

```
  BY index, remote_host, status, source  
| sort - count
```



The screenshot shows the Splunk Enterprise web interface. At the top, the search bar contains the query: `| tstats count WHERE index=dht11-dev BY index`. The results show 7,043 events. The 'Statistics (1)' tab is selected, displaying a table with two columns: 'Index' and 'count'. The table has one row with the value 'dht11-dev' and a count of '7043'.

Index	count
dht11-dev	7043

- Focus the event analysis on the highest-volume sources.

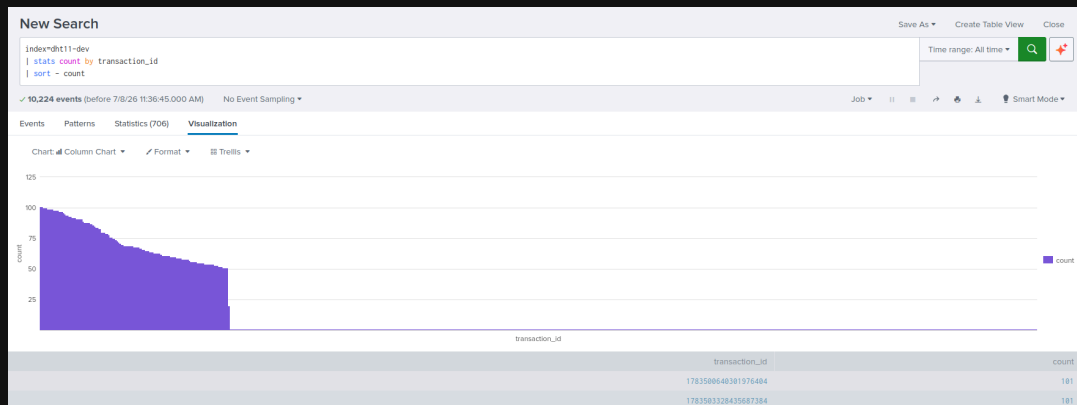
Query #2: Look for high events count on transaction

- Get a list of transactions grouped by event count

```
| tstats count WHERE index=your_index BY transaction_id  
| sort - count
```

If your app logs any `trx_id` kind of field (and it should), you can immediately answer: **which transaction types are generating the most lines?**

- Transactions that log way more (or less) events than average are a sign of abnormal behavior



Query #3: Pattern detection

Look for exact matches

```
index=your_index
| stats count by _raw
| sort -count
```

- By using `props.conf` and `transforms.conf` you can replace `_raw` with a more precise subset
- Exact match queries are fast and precise but do not look for patterns

Look for repeated patterns

```
index=your_index
| cluster t=0.9 showcount=true
| table cluster_count _raw
| sort - cluster_count
```

- Split results further for deeper analysis. Use them to isolate frequent events and patterns.
- `| cluster` groups similar log lines **automatically**, no regex needed.

DEMO!

In summary

What have we seen today?

Where are the **costs** in logging and how you can use Splunk to help you identify and reduce them.

1. Act at indexing time

Use structured logging and `props.conf` and `transforms.conf` to help splunk detect fields, route events and discard them

2. Analyze your logs and find insights on how to reduce them

Pick the highest logging index, aggregate by transactions, find patterns

3. Build alerts and dashboard

Operationalize findings so the whole team can monitor the logging trends

Thank you very much!

Michele Caci

